

Singly Linked List Operations

Create a **Singly Linked List** and implement the following operations:

1. Traversal

- Traverse the linked list and display all the elements.

2. Insertion

- Insert a new node at the **beginning** of the linked list.
- Insert a new node at the **end** of the linked list.
- Insert a new node at a **specific position** in the linked list.

3. Deletion

- Delete the node from the **beginning** of the linked list.
- Delete the node from the **end** of the linked list.
- Delete the node from a **specific position** in the linked list.

4. Searching

- Search for a given element in the linked list and determine whether it is present.

5. Updation

- Search for a given element and update its value with a new value.

6. Reversal

- Reverse the entire singly linked list and display the resulting linked list.

Creation

```
public class SinglyLinkedList {
    int val;
    SinglyLinkedList next;
    static SinglyLinkedList head;

    SinglyLinkedList(){

    }

    SinglyLinkedList(int val){
        this.val = val;
    }

    SinglyLinkedList(int val, SinglyLinkedList next){
        this.val = val;
        this.next = next;
    }
}
```

1. Traversal

```
private static void Traversal(SinglyLinkedList head){
    SinglyLinkedList curr = head;

    while(curr != null){
        System.out.print(curr.val + "->");
    }
}
```

```

        curr = curr.next;
    }
    System.out.println("null");
}

```

2. Insertion

2a. Insertion at the start

```

private static SinglyLinkedList InsertAtstart(SinglyLinkedList node){
    node.next = head;
    head = node;
    return head;
}

```

2b. Insertion at the end

```

private static SinglyLinkedList InsertAtEnd(SinglyLinkedList node){
    if(head == null){
        head = node;
        return head;
    }

    SinglyLinkedList curr = head;

    while(curr.next != null){
        curr = curr.next;
    }

    curr.next = node;
    return head;
}

```

2c. Insertion at the specific position

```

private static SinglyLinkedList InsertAtPosition(SinglyLinkedList node, int position){
    if(position == 1){
        node.next = head;
        head = node;
        return head;
    }

    SinglyLinkedList curr = head;
    int counter = 1;

    while(curr != null && counter < position - 1){
        curr = curr.next;
        counter++;
    }

    if(curr != null){
        node.next = curr.next;
        curr.next = node;
    }
}

```

```
    }else System.out.println("Position Not Found");

    return head;
}
```

3. Deletion

3a. Delete at Start

```
private static SinglyLinkedList DeleteAtStart() {
    if (head == null) return null;
    head = head.next;
    return head;
}
```

3b. Delete at End

```
private static SinglyLinkedList DeleteAtEnd(){
    if(head == null) return null;

    if(head.next == null){
        head = null;
        return head;
    }

    SinglyLinkedList curr = head;

    while(curr.next.next != null){
        curr = curr.next;
    }

    curr.next = null;
    return head;
}
```

3c. Delete at Position

```
private static SinglyLinkedList DeleteAtPosition(int position){
    if(head == null) return null;

    if(position == 1){
        head = head.next;
        return head;
    }

    SinglyLinkedList curr = head;
    int counter = 1;

    while(curr != null && counter < position - 1){
        curr = curr.next;
        counter++;
    }

    if(curr != null && curr.next != null){
```

```

        curr.next = curr.next.next;
    }else{
        System.out.println("Position Not Found");
    }

    return head;
}

```

4. Searching

```

private static boolean CheckIfPresent(int value){
    SinglyLinkedList curr = head;

    while(curr != null){
        if(curr.val == value){
            return true;
        }
        curr = curr.next;
    }

    return false;
}

```

5. Updation

```

private static SinglyLinkedList UpdateElement(int oldValue, int newValue) {
    SinglyLinkedList curr = head;

    while (curr != null) {
        if (curr.val == oldValue) {
            curr.val = newValue;
            break;
        }
        curr = curr.next;
    }

    return head;
}

```

6. Reverse

```

private static SinglyLinkedList ReverseList(SinglyLinkedList list){
    if(list == null || list.next == null) return list;

    SinglyLinkedList prev = null;
    SinglyLinkedList curr = list;

    while (curr != null){
        SinglyLinkedList temp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = temp;
    }
}

```

```
    return prev;
}
```

Main Function

Code

```
public static void main(String[] args){
    SinglyLinkedList one = new SinglyLinkedList(1);
    SinglyLinkedList two = new SinglyLinkedList(2);
    SinglyLinkedList three = new SinglyLinkedList(3);
    SinglyLinkedList four = new SinglyLinkedList(4);

    one.next = two;
    two.next = three;
    three.next = four;

    head = one;

    System.out.println("Printing the Singly Linked List");
    Traversal(head);

    SinglyLinkedList zero = new SinglyLinkedList(0);
    System.out.println("Inseting node 0 at start");
    InsertAtstart(zero);
    Traversal(head);

    SinglyLinkedList five = new SinglyLinkedList(5);
    System.out.println("Inserting node 5 at the end");
    InsertAtEnd(five);
    Traversal(head);

    SinglyLinkedList six = new SinglyLinkedList(6);
    System.out.println("Inserting node 6 at position 2");
    InsertAtPosition(six,2);
    Traversal(head);

    System.out.println("Deleting at Start");
    DeleteAtStart();
    Traversal(head);

    System.out.println("Deleting at Start");
    DeleteAtEnd();
    Traversal(head);

    System.out.println("Deleting at position 2");
    DeleteAtPosition(2);
    Traversal(head);

    System.out.println("Checking if 6 is presend or not :- "+CheckIfPresent(6));

    System.out.println("Checkig if 5 is present or :- " + CheckIfPresent(5));

    System.out.println("Updating node 3 value with 5");
    UpdateElement(3,5);
}
```

```

        Traversal(head);

        System.out.println("Printing the reverse list");
        SinglyLinkedList ReversedList = ReverseList(head);
        Traversal(ReversedList);

    }

```

Output

```

Printing the Singly Linked List
1->2->3->4->null
Inseting node 0 at start
0->1->2->3->4->null
Inserting node 5 at the end
0->1->2->3->4->5->null
Inserting node 6 at position 2
0->6->1->2->3->4->5->null
Deleting at Start
6->1->2->3->4->5->null
Deleting at Start
6->1->2->3->4->null
Deleting at position 2
6->2->3->4->null
Checking if 6 is presend or not :- true
Checkig if 5 is present or :- false
Updating node 3 value with 5
6->2->5->4->null
Printing the reverse list
4->5->2->6->null

```